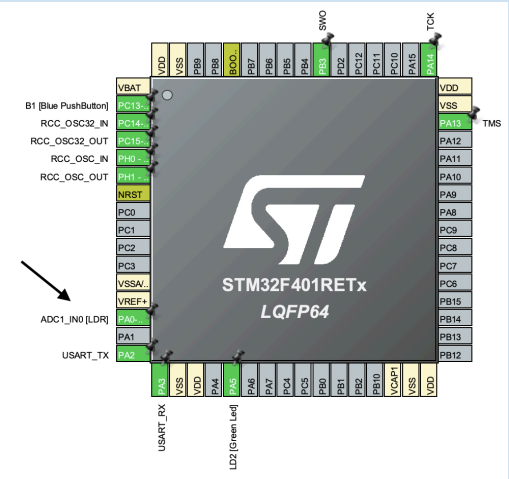
Mark	6/11

Team name:	B9		
Homework number:	Homework 06		
Due date:	28/10/2025		
Contribution:	No	Partial	Full
Pica Mirko			✅
Prendin Christian			✅
Pignataro Gabriele			✅
Moazenkakhki Mostafa			✅
Yarrabbotula Sunandha			✅
Notes:			

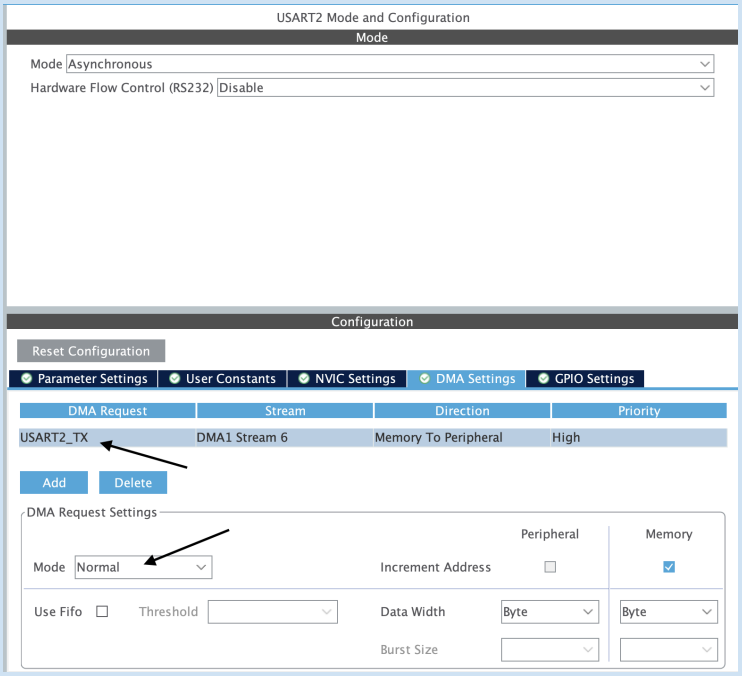
Project name	LDR		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			✅

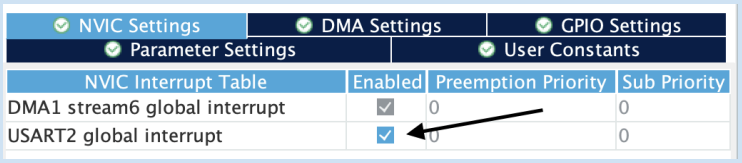
Part 1. Complete the ADC project 3a.
Report not required, because we have seen the solution in class.

Part 2. Complete the ADC project 3b (related to the LDR). Make sure to use the half complete callback.
From the “Hands-on Lab Schematics.pdf” file we found out that the LDR is connected to pin PA0, so we set it up as ADC1_IN0.

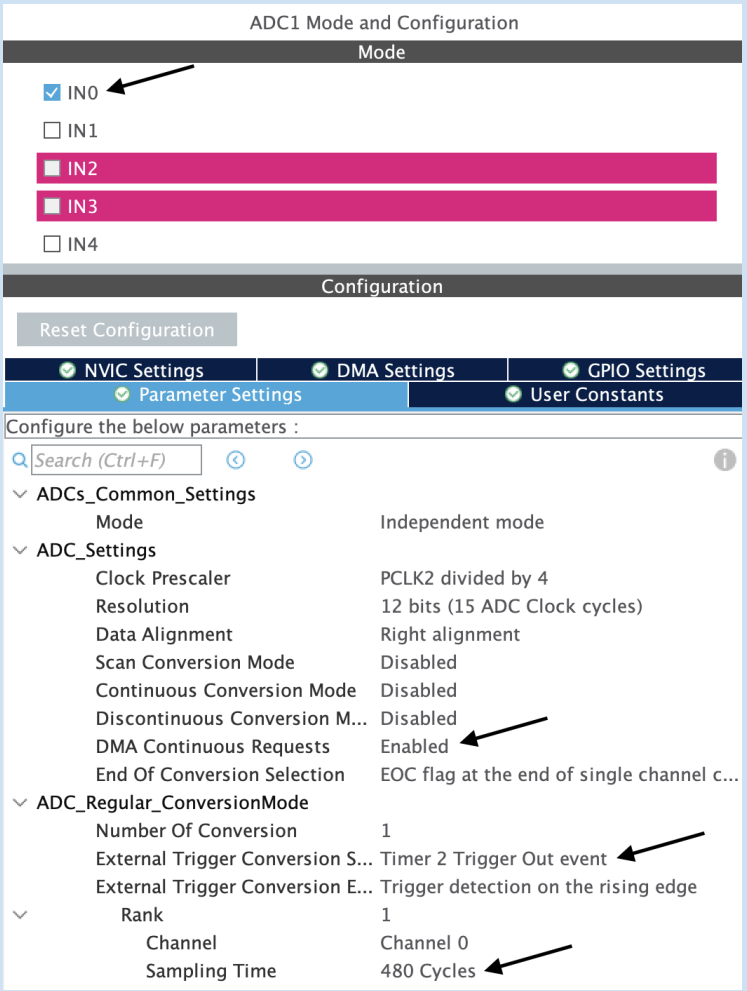


As in the previous homeworks we go into the connectivity section in USART2 and we enable the interrupt and the DMA in transmission.

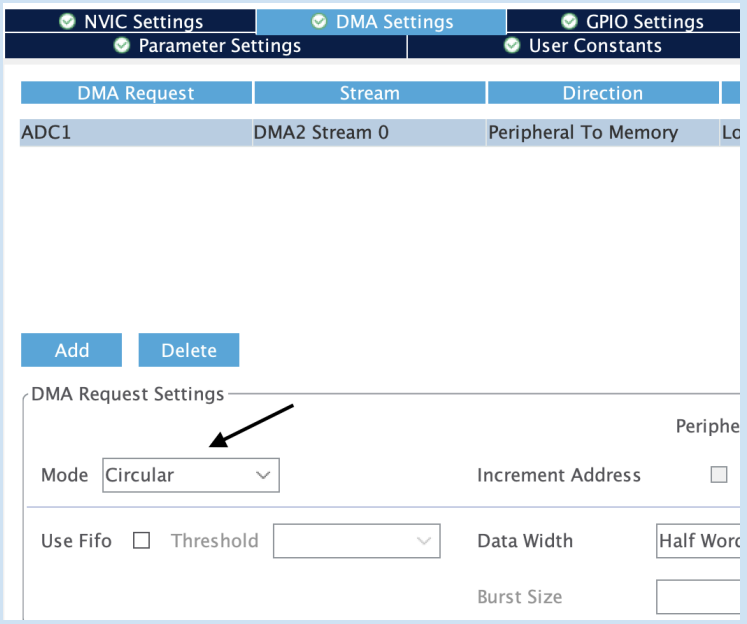




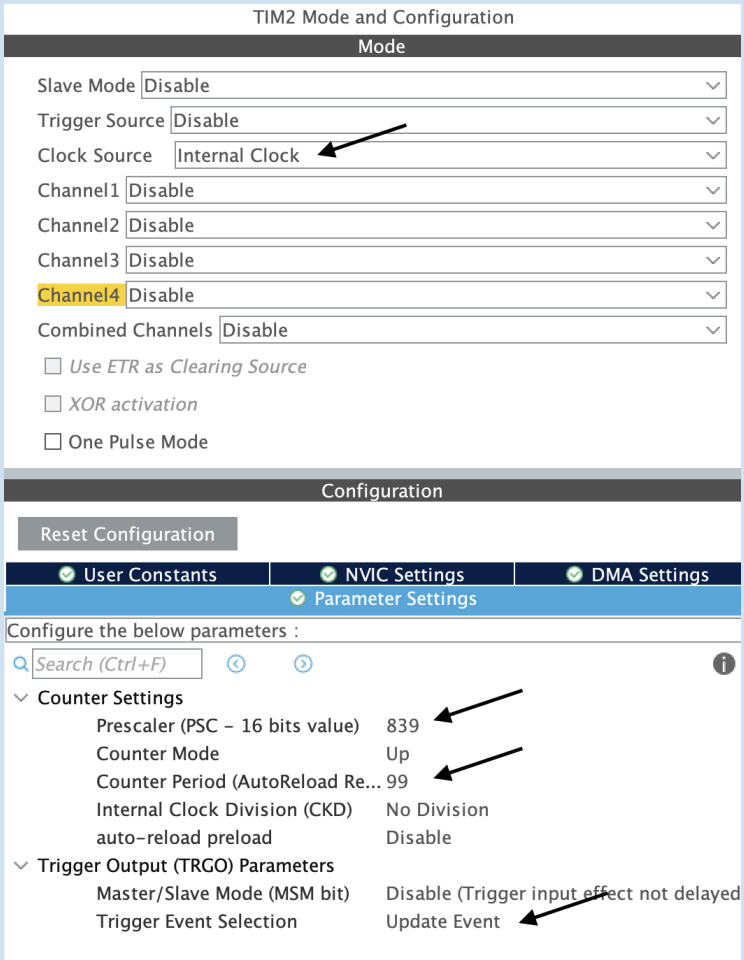
We now go into the analog section in ADC1, IN0 should be enabled (after enabling pin PA0). In the settings we leave everything default except for “DMA Continuous Request” to be enabled, “External Trigger Conversion Source” where we put “Timer 2 trigger out event” and “Sampling Time” where we put “480 cycles”. And of course we enable the ADC1 global interrupt.



Now we enable DMA for ADC1 and set the mode to “Circular”.



Now, we have to configure TIM2 to trigger every millisecond. We set the clock source, the prescaler at 839 and the counter at 99. This time we do not need the timer interrupt, instead we want to trigger the ADC so in “Trigger event selection” we put “Update event”.



Now let's modify our "main.c" file. We include "stdio.h" for the snprintf function and "math.h" for the pow function.

```
/* Private includes ----------
/* USER CODE BEGIN Includes */
#include "stdio.h"
#include "math.h"
/* USER CODE END Includes */
```

We declare two variables that we will use later. One is for storing the readings, the other for storing the half complete value.

```
/* USER CODE BEGIN PV */
uint16_t adc_dma_buffer[1000];
float first_half_resistance_avg;
/* USER CODE END PV */
```

In the main function we just need to start the ADC and the timer.

```
/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, adc_dma_buffer, 1000);
HAL_TIM_Base_Start(&htim2);
/* USER CODE END 2 */
```

We now write the two callback functions. The half complete calculates the resistance average for the first 500 readings. Then the complete does the rest and calculates the lux value sending it to the remote terminal.

```c
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef* hadc) {
    uint32_t sum = 0;
    for (int i = 0; i < 500; i++) {
        sum += adc_dma_buffer[i];
    }
    float average_adc = sum / 500;
    float voltage = average_adc * 3.3 / 4095;
    first_half_resistance_avg = (voltage * 100000) / (3.3 - voltage);
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc) {
    uint32_t sum = 0;
    for (int i = 500; i < 1000; i++) {
        sum += adc_dma_buffer[i];
    }
    float average_adc = sum / 500;
    float voltage = average_adc * 3.3 / 4095;
    float second_half_resistance_avg = (voltage * 100000) / (3.3 - voltage);

    float resistance_avg = (first_half_resistance_avg + second_half_resistance_avg) / 2;
    float lux = pow(10 * (100000 / resistance_avg), 1.25);

    char string[32];
    int length = snprintf(string, sizeof(string), "LUX: %.2f\n\n", lux);
    HAL_UART_Transmit_DMA(&huart2, (uint8_t *)string, length);
}
/* USER CODE END 0 */
```

As said in class, probably in this homework it's not necessary to split the calculations to get the correct value, but in other cases it might be important. We tested our code and it works.

Professor comments:	